

Vehicles Recommendation Web



universidad
de león

The Team:

- ★ Jimena Onrubia Álvarez
- ★ Andrés Martínez Colina
- ★ Ignacio Gil Rojo

1. Introduction

➤ Product Description

A web app that combines an AI-powered chatbot (Groq) with visual filters to recommend vehicles matching your lifestyle and preferences. Save favorites to your account and find real car images from the internet.

➤ Problem Solved and Project Focus

➤ Target Audience

2. Team & Methodology

The project was built collaboratively, with each member sharing development responsibilities while owning a specific role: **Jimena** (Project Coordinator & Community Manager) led task management and milestone tracking; **Andrés** (Developer & QA Lead) handled testing, validation, and quality assurance; and **Ignacio** (Developer & Documentation Lead) managed project documentation.

The team followed an agile, Scrum-based workflow with short sprints. Jimena defined objectives and distributed tasks each sprint, while coordination and issue resolution happened through regular WhatsApp meetings.

3. Requirements

Functional Requirements

1. The application must continuously update the SQL database in the background, periodically modifying selected values to simulate live system activity and data changes.
2. The application must store and manage registered user accounts securely, including user profile data and authentication-related information.
3. The application must store and manage a list of cars and models marked by each user as favorites.
4. The application must provide filtering capabilities (e.g., brand, model, year, price range, fuel type, and other relevant attributes) for browsing and searching vehicles.
5. The application must provide access to an intelligent chatbot capable of answering user questions and assisting users with recommendations, navigation, and support.
6. The application must operate smoothly and responsively under normal and expected workloads.
7. The application must be deployed on the Internet and accessible through a public URL.

4. Design and Architecture

The system is based on Clean Architecture, structured into Domain, Application (Ports and Adapters), and Infrastructure layers. The Domain defines the core business entities and rules, while Ports and Adapters act as interfaces between the core logic and external systems such as the database and user interface. The Infrastructure layer provides concrete implementations, including persistence. A Singleton pattern is used to manage the database connection, ensuring a single shared instance throughout the application.

5. Development and Implementation

Development started with the SQL database, including the design of tables and the creation of Go tools to convert and adapt data from CSV into structured SQL format. In the same stage, a Go-based simulation program was implemented to emulate external API behavior by continuously updating database values.

Afterwards, the backend was developed in Go using the Visual Studio Code extension, implementing the main REST API, business logic, authentication, and system operations such as vehicles, users, favorites, and chatbot integration.

Finally, the frontend was developed using Vue.js, providing the user interface for browsing vehicles, applying filters, managing favorites, and interacting with the chatbot. The system was then integrated and unit tested to ensure correct communication between all components.

6. CI/CD & Deployment

7. Final Results & Screenshots

8. Conclusions & Lessons Learned

9. Relevant Links

GitHub:

<https://github.com/isw2-unileon/project-VehiclesRecomendationWeb.git>

Internet URL: